Homework 5

CS Osvita: from C to GC

February 11, 2026

This homework is designed to achieve two objectives:

1. Cover enough C syntax that we can later, in the course, disassemble short C snippets to better understand the hardware/software interface.

2. Increase your familiarity with a garbage collector (GC). You will learn basic GC strategies and explore the design details of GCs in multiple languages.

While not necessary, it's useful to have a copy of The C Programming Language (also known as K&R C) available as a reference. It will be useful to read Chapters 5 and 6.

1 Pangram

In this problem, you are provided with a Python program that checks whether a given string is a pangram. A pangram is a string that contains every letter from a to z at least once.

Your task is to write an equivalent C program that performs the same pangram check. The goal is to compare the runtime performance and overhead of the Python interpreter versus a compiled C program. This exercise will also be an introduction to working with pointers, as you will handle C-style strings represented by char*.

Additionally, this task will highlight how the constraints of C often lead to more efficient memory and performance-conscious solutions compared to high-level languages like Python. So, implement a C program that checks if a given string is a pangram. Optimize your solution to be space-efficient, as discussed in the lecture. You can run and test the Python version like this:

```
./test.sh python3 ./find_pangrams.py
```

2 Write Garbage Collector

This task involves a significant amount of C. Before diving in, we recommend reading the following article to build a strong foundation.

Automatic memory management via garbage collection frees programmers from worrying about details of when to call $malloc$ and $free$ and eliminates an entire category of errors. However, garbage collection can be a source of many

performance issues. Understanding the behavior of your language's garbage collector can help you build systems with lower latency and higher throughput.

2.1 Why?

Garbage collection makes a programming language appear to have infinite memory by automatically reclaiming unused memory. When memory is low, the system identifies "garbage," previously allocated memory no longer in use, and frees it up.

For garbage collection to be effective, the system must ensure that only memory that can't be accessed again is reclaimed. An object is considered "in use" if:

- It's referenced by a variable still in scope.
- It's referenced by another in-use object.

This creates a graph of reachable objects. Any object not in this graph is considered dead and can be safely collected.

2.2 Task

Your **goal** is to complete the implementation of the mark and sweep functions to ensure that all tests pass. Before making any changes, ensure you fully understand the implementation. You should be able to describe:

- The policy for deciding when garbage collection happens
- What causes calls to $malloc$ and $free$
- How the "mark-and-sweep" algorithm works. You might notice that the "mark-and-sweep" algorithm is very similar to a graph search algorithm you're already familiar with - DFS.

If you'd prefer a more visual presentation of the mark-and-sweep algorithm, you might also find it helpful to consult section 9.10 (Garbage Collection) of CS:APP.

Finally, add some instrumentation code to measure and print out the following:

- How often the garbage collector runs
- How many objects are marked/swept on each run

What happens to the measured values if you change:

- Usage pattern. The existing $perfTest$ function has a simple pattern with a sequence of $pushInt$ followed by a sequence of pop. You might also consider interleaved $pushInt$ and pop (with different numbers of $pushInt$ for each pop), or some even more complicated patterns involving $pushPair$.

- Parameters (max stack size, initial threshold, resizing factor)
- Policy for when to run garbage collection

Based on your measurements, what can you say about the overhead of garbage collection (at least in the context of this minimal example)? What sort of performance tradeoffs are available?

2.3 Implementation details

Imagine we're writing an interpreter for a small, dynamically typed language with two object types: ints and pairs. Here's an enum to identify an object's type:

```
typedef enum {
  OBJ_INT,
  OBJ_PAIR
} ObjectType;
```

A pair can contain any combination of ints and pairs. This flexibility allows for complex structures. In the Virtual Machine, objects are implemented using a tagged union in C:

```
typedef struct sObject {
  ObjectType type;

  union {
    /* OBJ_INT */
    int value;

    /* OBJ_PAIR */
    struct {
      struct sObject* head;
      struct sObject* tail;
    };
  };
} Object;
```

The main Object struct has a *type* field to identify if it's an *int* or a *pair*. It uses a *union* to store the data, as a *union* allows overlapping fields in memory. Since an object is either an *int* or a *pair*, the *union* efficiently uses memory by not allocating space for both types simultaneously.

Now we can use this datatype in a virtual machine (VM). The VM maintains a stack to store variables currently in scope. Most language VMs, whether stack-based (like JVM and CLR) or register-based (like Lua), use a stack for local and temporary variables. We can model this explicitly and simply:

```c
#define STACK_MAX 256
typedef struct {
  Object* stack[STACK_MAX];
  int stackSize;
} VM;
```

With our basic data structures ready, let's write a function to create and initialize a VM:

```c
VM* newVM() {
  VM* vm = malloc(sizeof(VM));
  vm->stackSize = 0;
  return vm;
}
```

Once we have a VM, we need to be able to manipulate its stack:

```c
void push(VM* vm, Object* value) {
  assert(vm->stackSize < STACK_MAX, "Stack overflow!");
  vm->stack[vm->stackSize++] = value;
}

Object* pop(VM* vm) {
  assert(vm->stackSize > 0, "Stack underflow!");
  return vm->stack[--vm->stackSize];
}
```

Now that we can store items in "variables," we need to create objects. Here's a helper function to do that:

```c
Object* newObject(VM* vm, ObjectType type) {
  Object* object = malloc(sizeof(Object));
  object->type = type;
  return object;
}
```

Using that, we can write functions to push each kind of object onto the VM's stack:

```c
void pushInt(VM* vm, int intValue) {
  Object* object = newObject(vm, OBJ_INT);
  object->value = intValue;
  push(vm, object);
}

Object* pushPair(VM* vm) {
  Object* object = newObject(vm, OBJ_PAIR);
```

```
  object->tail = pop(vm);
  object->head = pop(vm);

  push(vm, object);
  return object;
}
```

That's it for our little VM. With a parser and interpreter using these functions, we'd have a complete language. However, since we don't have infinite memory, we need to implement the mark-and-sweep-garbage collection next.

2.4 Mark

The first phase is marking. We need to walk all of the reachable objects and set their mark bit. The first thing we need then is to add a mark bit to Object:

```
typedef struct sObject {
  unsigned char marked;
  /* Previous stuff... */
} Object;
```

When we create a new object, we modify newObject() to initialize marked to zero. To mark all of the reachable objects, we start with the variables that are in memory, which means walking the stack:

```
void markAll(VM* VM) {
  for (int i = 0; i < vm->stackSize; i++) {
    mark(vm->stack[i]);
  }
}
```

2.5 Sweep

The next phase is to sweep through all of the objects we've allocated and free any of them that aren't marked. But there's a problem here: all of the unmarked objects are, by definition, unreachable! We can't get to them!

The VM has implemented the language's semantics for object references, so we're only storing pointers to objects in variables and the pair fields. As soon as an object is no longer pointed to by one of those, the VM has lost it entirely and leaks memory.

The trick to solve this is that the VM can have its references to objects that are distinct from the semantics that are visible to the language user. In other words, we can keep track of them ourselves.

The simplest way to do this is to just maintain a linked list of every object we've ever allocated. We extend Object itself to be a node in that list:

```c
typedef struct sObject {
  /* The next object in the list of all objects. */
  struct sObject* next;

  /* Previous stuff... */
} Object;
```

The VM keeps track of the head of that list:

```c
typedef struct {
  /* The first object in the list of all objects. */
  Object* firstObject;

  /* Previous stuff... */
} VM;
```

In newVM() we make sure to initialize firstObject to NULL. Whenever we create an object, we add it to the list:

```c
Object* newObject(VM* vm, ObjectType type) {
  Object* object = malloc(sizeof(Object));
  object->type = type;
  object->marked = 0;

  /* Insert it into the list of allocated objects. */
  object->next = vm->firstObject;
  vm->firstObject = object;

  return object;
}
```

This way, even if the language can't find an object, the language implementation still can. Congratulations! We have a garbage collector!

Further resources

Understanding Allocations: the Stack and the Heap talks about when a variable "escapes to the heap" (and must be garbage collected).

These case studies describe efforts to optimize garbage collection in production Go systems:

- Taming Go's Memory Usage, or How We Avoided Rewriting Our Client in Rust
- Go memory ballast: How I learnt to stop worrying and love the heap

- How We Saved 70K Cores Across 30 Mission-Critical Services and the related video Large-Scale, Semi-Automated Go Garbage Collection Tuning at Uber

The following articles/talks describe garbage collection strategies in other languages:

- Python:

  Garbage Collector Design

  The Memory Management chapter of CPython Internals

- JavaScript:

  Most of the posts from the V8 blog with the "memory" tag

- Ruby:

  A Day in the Life of a Ruby Object

  Chapter 5 of Ruby Hacking Guide

The Go website describes Diagnostics for profiling, tracing, debugging, and runtime statistics/events.

Visualizing Garbage Collection Algorithms provides some helpful animations.

Crafting Interpreters includes a chapter on Garbage Collection, in the context of implementing the clox runtime.

Section 17 of Alex Aiken's compilers course gives a more theoretical perspective on garbage collection.

Ardan Labs has an extensive reading list on garbage collection, including the series Garbage Collection in Go which gives a very practical perspective.